Foruster: A Cross-Platform Tool for Live Forensic Triage and Anomaly Detection

Marcos Jesús Sequera Fernández[a], Mohammadhossein Homaei[b], Óscar Mogollón Gutierrez[c], José Carlos Sancho Núñez[d]

[a] *marcosjesus@unex.es*

[b] *mhomaein@alumnos.unex.es*

[c] *oscarmg@unex.es*

[d] *jcsanchon@unex.es*

Abstract

Foruster is a cross-platform desktop application, developed in Rust, for live-system forensic analysis. Unlike traditional tools that require system shutdown, Foruster is designed to identify and catalog files of interest on active storage volumes. Its user interface, built with the Slint framework, guides the analyst through the selection of devices, the configuration of search profiles, and the real-time visualization of results. The software features heuristic detection of anomalies, such as deceptive file extensions, and ensures the integrity of findings through cryptographic hashing, optimizing the digital forensic investigation process.

Keywords: live-system forensics, forensic software, data extraction, Rust, digital security, incident response

---

Table 1: Code metadata

Nr.	Code metadata description	Metadata
C1	Current code version	1.0.0
C2	Permanent link to code/repository	`https://github.com/Grupo-de-Ingenieria-de-Medios-GIM/foruster`
C3	Permanent link to Reproducible Capsule	–
C4	Legal Code License	GPLv3
C5	Code versioning system used	git
C6	Software code languages, tools, and services	Rust, Slint
C7	Compilation requirements, operating environments & dependencies	Rust 1.89 or later; Windows 10/11, Linux (Ubuntu 20.04+)
C8	Link to developer documentation/manual	`https://github.com/Grupo-de-Ingenieria-de-Medios-GIM/foruster/blob/main/README.md`
C9	Support email for questions	marcosjesus@unex.es

1. Motivation and Significance

Traditional digital forensics methodology has historically centered on post-mortem analysis, which involves creating bit-by-bit disk images from powered-off systems to preserve evidence integrity. However, this "dead-box" approach presents significant limitations in modern computing environments. A primary drawback is the loss of volatile data—such as running processes, active network connections, or encryption keys stored in RAM—which is irretrievably lost upon shutdown and often contains crucial information for contextualizing a security incident [1, 2].

The widespread use of full-disk encryption further compounds this issue. While a system is operational, encrypted volumes are mounted and their data is accessible. Once powered off, accessing this data becomes considerably more complex, if not impossible, without the decryption keys, which often only reside in volatile memory [3]. Furthermore, the exponential growth in data volume on modern storage devices renders full imaging a time- and resource-intensive process, delaying response capabilities in time-sensitive investigations and demanding significant storage capacity [4, 5].

To overcome these obstacles, live forensics has emerged as a complementary paradigm. This technique allows investigators to collect volatile data and analyze systems while they are running, facilitating access to mounted encrypted volumes [6]. Nonetheless, live analysis introduces its own challenges, primarily the unavoidable alteration of the system's state—a factor that must be meticulously managed and documented to maintain evidentiary validity [7]. Moreover, many existing live analysis tools require a high degree of technical expertise and often rely on command-line interfaces, which can limit their adoption.

The current ecosystem of forensic tools tends towards specialization: on one

hand, tools like the Volatility Framework focus exclusively on memory analysis [8], while platforms such as Autopsy or EnCase are designed primarily for post-mortem analysis of disk images. Although tools with live acquisition capabilities exist, such as FTK Imager, they often lack integrated functionalities for anomaly detection. Commercial solutions are typically costly and platform-specific, while open-source alternatives may present performance or compatibility trade-offs.

This landscape reveals a functional gap: the need for a tool that combines file triage on live systems with anomaly detection, packaged in a memory-safe, cross-platform, and open-source solution. Foruster was developed to address this void. Its implementation in Rust leverages the language's memory safety guarantees to prevent common vulnerabilities, such as buffer overflows, that could compromise the integrity of the forensic tool itself [9]. Its asynchronous architecture and graphical user interface aim to balance performance with accessibility.

The impact of a tool like Foruster is significant for incident response teams, security analysts, and the research community. By being open-source, it promotes transparency and reproducibility, allowing its methods to be scrutinized and validated by the community—a key tenet for the admissibility of digital evidence under rigorous legal standards like the Daubert standard [1].

2. Software description

Foruster is a desktop software that works on multiple platforms and is written in Rust. It allows live-system file investigation without needing to stop the system. Unlike usual forensic tools, Foruster works directly on active storage to find and record files that may be important, while the computer keeps running. The interface uses the Slint framework [10] and helps users choose devices, configure search settings, and see results immediately. The program also uses heuristic checks to find files with misleading extensions and protects evidence integrity with cryptographic hashes.

2.1. Software Architecture

Foruster has a modular design organized as a Rust workspace, separating functionalities into different crates to facilitate maintenance and improve performance (Figure 1). This layered structure promotes independence and code reuse across platforms, and its key components are described in Table 2.

2.2. Core Analysis Algorithm

The scanning and anomaly detection workflow must balance speed and completeness. To achieve this, a set of visited paths is maintained to prevent

Table 2: Architectural Components

Component	Description
`foruster-storage`	The base layer that abstracts storage operations. On Windows, it uses Win32 APIs (e.g., `FindFirstVolumeW`, `DeviceIoControl`), while on Linux, it reads from virtual filesystems like `/sys/block` and `/proc/self/mountinfo`. This ensures consistent behavior while respecting platform-specific conventions.
`analysis`	Contains the core logic for file system traversal. It runs on the Tokio async runtime [11] and uses the `async-walkdir` crate for parallel processing, ensuring the user interface remains responsive during long scans.
`profiling`	Implements the rule engine for file matching. It allows defining profiles based on MIME types, extensions, or predefined categories (e.g., documents, images). Users can leverage default profiles or create custom filters.
`file-format`	A utility crate responsible for identifying file types by reading their headers (magic numbers), a foundational technique in file system forensics [12]. This is crucial for detecting extension mismatches.
`api`	Acts as a bridge, safely connecting the backend logic (analysis and storage) with the frontend graphical interface.
`desktop`	The presentation layer, which implements the graphical user interface (GUI) using Slint widgets [10]. This approach avoids heavy frameworks like Electron, resulting in a lightweight, native application.

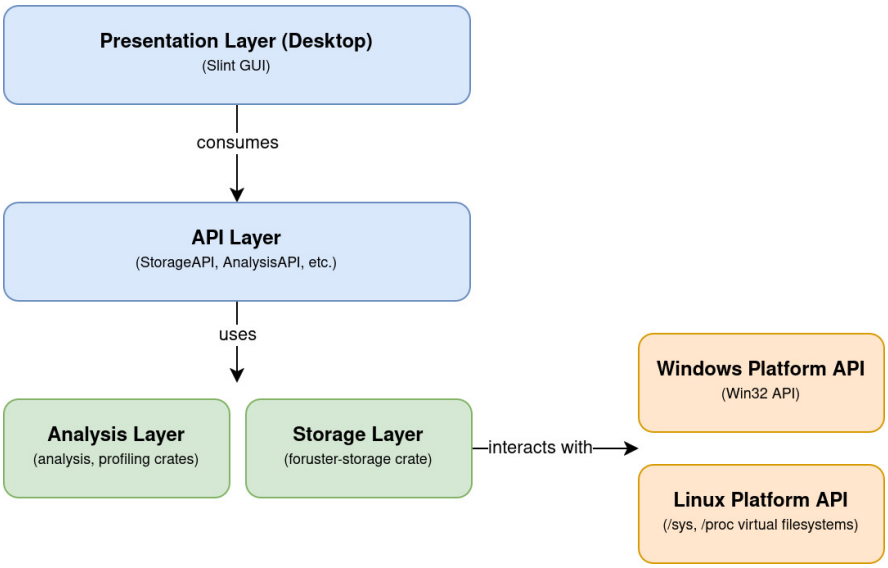


Figure 1: Modular architecture showing layers from user interface to operating system interaction.

infinite loops caused by symbolic links. The process uses a breadth-first search, which allows the scan to be stopped early if necessary while still providing partial results. Hashes (MD5, SHA-256, BLAKE3) are computed selectively for relevant or suspicious files to conserve resources. The detection logic is designed to allow legitimate polyglot files (like Office documents that are also ZIP archives), minimizing false positives. The workflow is formally described in Algorithm 1, which details the `analysis::walker` function.

2.3. Forensic Integrity and Chain of Custody

Foruster follows ISO/IEC 27037:2012 [13] for digital evidence. In live analysis scenarios ("hot work"), the methodology must minimize the footprint on the system, preserve file metadata, and ensure traceability through a robust chain of custody. Foruster has been designed to operate within this rigorous framework, structuring its functionalities around the forensic phases of identification, acquisition, preservation, and presentation as shown in Figure 2.

2.3.1. Identification and Acquisition Phase

This phase focuses on locating and collecting potential evidence. Foruster addresses this by allowing a reproducible selection of the analysis scope, where the investigator defines the storage devices and search profiles. The analysis engine (analysis::walker) operates on the file system layer, which minimizes direct interaction with the hardware and reduces the system's footprint. This approach allows the acquisition of data from encrypted volumes (BitLocker, LUKS) that are mounted, a common scenario in active systems. Additionally, Foruster implements heuristics for identifying anomalies, such as discrepancies between the actual file type (verified using "magic numbers" with the file-format crate [12]) and its extension, thus uncovering evidence that may have been intentionally obfuscated.

2.3.2. Preservation and Presentation Phase

To ensure that the evidence is not altered or contaminated, the preservation phase is critical. Foruster ensures the integrity of each identified file by generating multiple cryptographic hashes. For each relevant file, the software calculates its MD5, SHA-256, and BLAKE3 hashes [14]. This set of hashes acts as a unique digital signature of the file's state at the exact moment of acquisition. This process is fundamental for the presentation of the evidence, as it establishes an auditable chain of custody. Any subsequent verification can recalculate these hashes to confirm that the presented evidence is an exact copy of the original.

2.3.3. Presentation Phase

The presentation of evidence must be clear, verifiable, and adhere to recognized standards. Foruster fulfills this phase in two ways. First, through its graphical interface, which allows the analyst a real-time visualization and filtering of the findings. Second, and more formally, by generating a final report in PDF format through the genpdfi crate. This document acts as a signed manifest of the extraction: it includes a detailed record of the operations, the analysis configuration, the characteristics of the investigated

disks, and, crucially, a list of all relevant files along with their cryptographic hashes. This final report constitutes a transparent and verifiable artifact that supports the presentation of evidence in a potential judicial context.

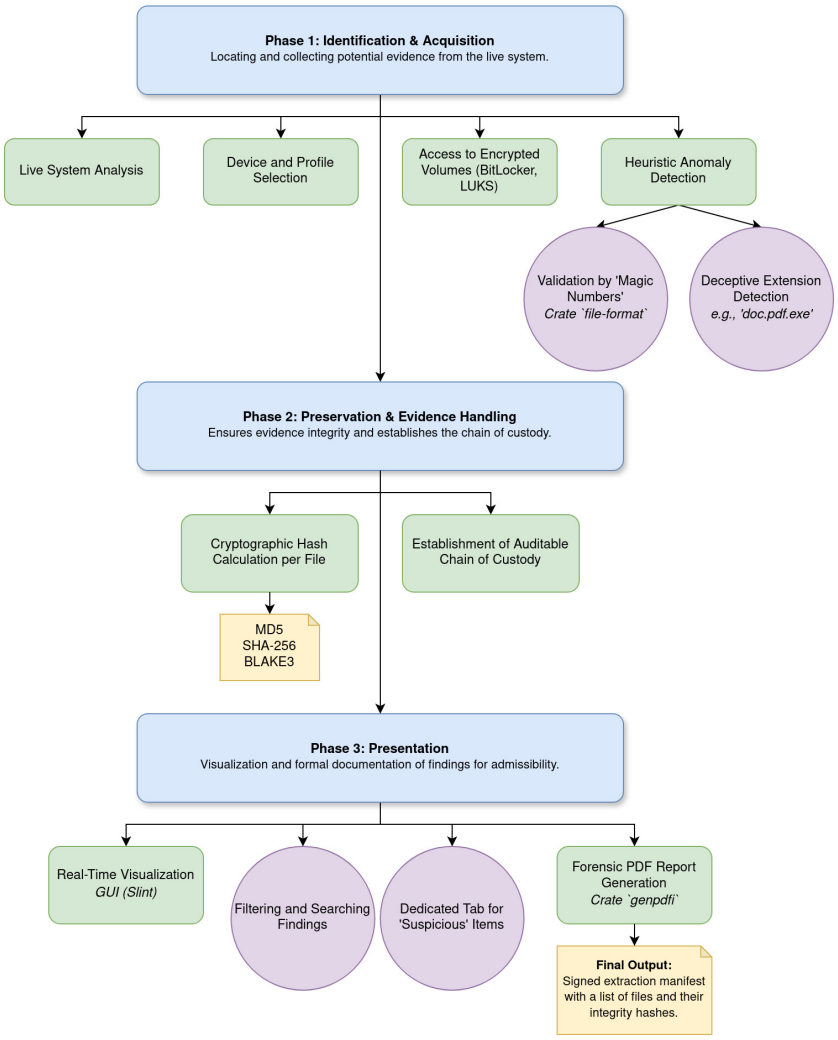


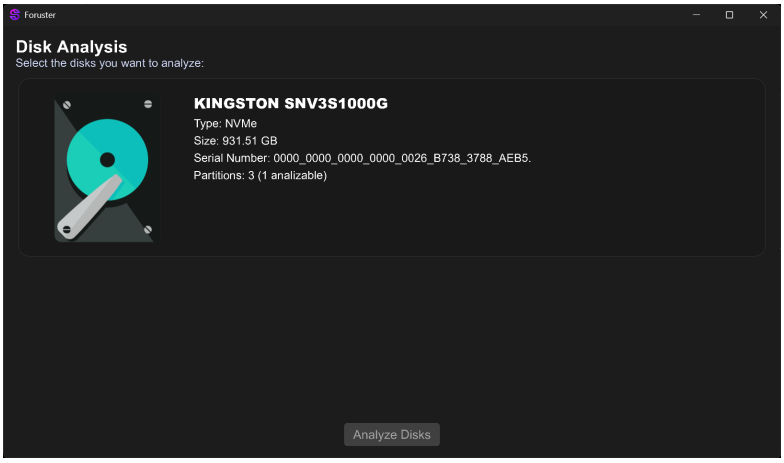
Figure 2: Diagram of Foruster's related forensic phases

2.4. Implementation Considerations

Rust's memory safety prevents crashes like buffer overflow or use-after-free. Async design with Tokio reduces CPU load on user tasks. Performance tests show less than 10% slowdown while scanning. Cross-platform build produces native Windows and Linux binaries from one codebase. Slint gives native GUI on all platforms without heavy frameworks.

3. Illustrative Examples

To show how Foruster works in practice, we present a typical incident response in a company setting. Suppose a forensic analyst needs to check an employee's workstation for possible confidential data copied to an external drive, without stopping normal work or alerting the user. The user interface workflow is shown in Figures 3 and 4.

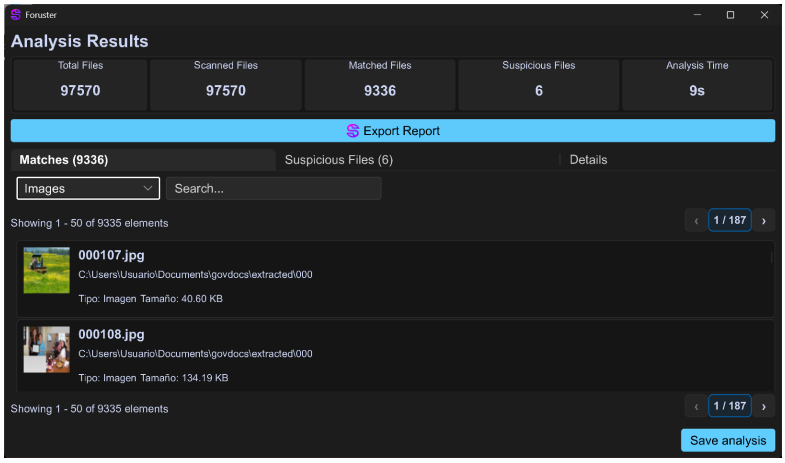


(a) Device selection screen.

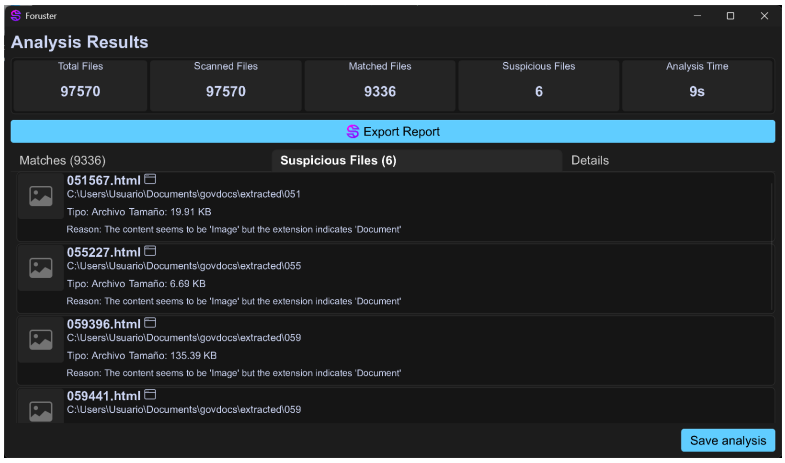
(b) Search profile configuration.

Figure 3: Foruster setup workflow. (a) The analyst selects the storage devices to be scanned. (b) The relevant search profiles are enabled to define the scope of the analysis.

The analyst opens Foruster on the live workstation. The software displays all connected storage devices (Figure 3a). In this case, the analyst selects the primary KINGSTON NVMe solid-state drive for analysis. Next, from the



(a) Real-time display of matches.



(b) Detection of anomalies.

Figure 4: Foruster analysis results view. (a) The main tab displays all files matching the selected profiles. (b) The "Suspicious" tab isolates files with anomalies, such as mismatched extensions.

profiles screen (Figure 3b), the analyst enables the "Videos", "Images", and "Audios" profiles to search for multimedia files that may have been exfiltrated.

When the scan begins, the interface updates in real-time. The analysis is extremely rapid, processing all 97,570 files on the target volume in just 9 seconds. The results screen (Figure 4a) shows that 9,336 files matched the selected profiles and 6 files were flagged as suspicious.

In the "Suspicious" tab (Figure 4b), the analyst examines the flagged files. For instance, a file named 051567.html has been identified as an anomaly.

Its extension suggests it is a document, but Foruster's file header analysis reveals that its actual content is an image (image/gif). The reason provided is: "The content seems to be 'Image' but the extension indicates 'Document'." This discrepancy suggests an intentional effort to hide a file, a common anti-forensic technique to evade detection.

The analyst can then select any of these files to view detailed metadata, including creation and modification timestamps, size, and the three cryptographic hashes (MD5, SHA-256, and BLAKE3) calculated by Foruster. These hashes are crucial for verifying the file's integrity and maintaining the chain of custody.

Finally, the analyst clicks the "Export Report" button to generate a forensic report in PDF format. This document, created automatically by the software, includes the analysis time, device information, the profiles used, a summary of the findings (total files, matches, suspicious files), and a detailed list of all relevant files with their full paths and cryptographic hashes. This report serves as a formal audit trail for management or potential legal proceedings.

4. Impact

The impact of Foruster is demonstrated through its performance efficiency, its advantages over existing tools, and its practical and scientific contributions to the digital forensics community. This section presents a quantitative evaluation of the tool's performance characteristics, a qualitative comparison with established alternatives, and a discussion of its overall significance.

4.1. Performance Evaluation

We evaluated Foruster's performance and accuracy across multiple hardware platforms and operating systems to establish a quantitative baseline. Tests were conducted on high-end (Intel i9-14900KF), mid-range (AMD Ryzen 5 7600X), and mobile (AMD Ryzen 7 PRO 7840U) systems using the standardized Govdocs1 corpus [15] for benchmarking.

As detailed in Table 3, Foruster's performance scales effectively with hardware and is significantly influenced by the operating system. On identical high-end hardware, the tool ran approximately 75% faster on PopOS than on Windows 11, highlighting the efficiency of Linux's file I/O for this task. The tool remains highly effective even on mobile hardware, ensuring viability for in-the-field triage. Across all scenarios, resource consumption was minimal, demonstrating a low system footprint.

The anomaly detection engine's effectiveness was validated on a ground-truth dataset of 500 files [16] with intentionally mismatched extensions. The engine successfully identified all 500 files, achieving a perfect **100% recall** rate for this common anti-forensic technique.

Table 3: Foruster performance benchmark using the Govdocs1 dataset.

Platform	**OS**	**Files/sec**	**Time (s)**	**RAM (MB)**
High-End Desktop (i9)	Windows 11	*3,322*	*29.38*	*172*
High-End Desktop (i9)	PopOS	*5,819*	*16.94*	*136*
Mid-Range Desktop (Ryzen 5)	CachyOS	*2,757*	*35.75*	*193*
Laptop (Ryzen 7)	NixOS	*2,269*	*43.43*	*228*

4.2. Comparative Analysis

Table 4 compares Foruster to other forensic tools. Foruster stands out with live file triage and anomaly detection in an open-source, cross-platform package. FTK Imager is good for imaging but cannot analyze live files. Autopsy, a widely used open-source platform [12], is for post-mortem cases with limited live support. Volatility [8] focuses on memory and needs extra tools for files. EnCase is enterprise-level but Windows-only and commercial.

Table 4: Comparison with existing tools

Feature	**Foruster**	**FTK Img.**	**Autopsy**	**Volatility**	**EnCase**
Live File Triage	✓	×	Limited	×	✓
Cross-Platform	✓	×	✓	✓	×
Encrypted Vol. Access	✓*	×	×	✓**	✓**
Anomaly Detection	✓	×	Manual	×	Limited
Open Source	✓	×	✓	✓	×
Memory Safety	✓	–	–	–	–
GUI	✓	✓	✓	×	✓
Cost	Free	Free	Free	Free	Commercial

*Requires mounted volumes
**Needs memory key extraction

4.3. Scientific and Practical Impact

Foruster fills a gap in forensic work by letting analysts quickly examine live systems before doing full imaging. This is useful in three cases: encrypted storage, where it reads mounted volumes without complex key work; time-sensitive incidents, where immediate triage informs decisions; and limited-resource environments, where selective scanning saves storage and time.

It also helps researchers study how evidence changes over time. By looking at live analysis compared to full disk images, investigators can see what data disappears after shutdown, which gives better insight into preserving evidence.

Being open-source under GPLv3 makes the tool transparent. Many closed-source programs hide their inner workings, which can cause problems in courts that follow standards like Daubert. With open code, anyone can check the methods and reproduce results. GitHub activity shows that both researchers and practitioners are actively contributing.

Although not widely adopted yet, Foruster is already used in some labs and security operations centers for quick triage. Its open-source license makes it easier for organizations that cannot afford commercial software.

Future work includes machine-learning detection for more complex obfuscation, distributed analysis for many endpoints, CASE model integration for tool interoperability [17], and extension to cloud and mobile systems. These updates will expand Foruster's reach while keeping live forensic capability at the core.

5. Conclusions

This paper presented Foruster, an open-source tool for live file triage on active systems. It addresses gaps in traditional forensics by allowing quick file discovery without shutting down the system. Key features include access to mounted encrypted volumes without complex key extraction, heuristic checks for suspicious files, a memory-safe Rust core, and automatic documentation with cryptographic hashes and PDF reports.

In tests, Foruster handled 2,269–5,819 files per second. Its extension-based anomaly detection achieved a 100% recall rate, confirming that checking file headers against extensions works reliably. System impact was low, so it can operate discreetly during investigations.

There are some limitations. The tool requires the system to be powered on with all volumes mounted, and its heuristics may fail to detect advanced obfuscation techniques such as steganography. It does not yet support CASE standards or cloud and mobile storage.

Future work will focus on adding machine learning for smarter anomaly detection, distributed deployment for larger networks, CASE support, and cloud/mobile integration. These updates aim to keep Foruster reliable, open, and accessible to organizations with limited budgets.

By offering an open, community-verified tool, Foruster helps advance live digital forensics and supports reproducible research while lowering barriers for smaller organizations.

Data Availability

The source code and documentation are openly available at the Foruster project repository: `https://github.com/Grupo-de-Ingenieria-de-Medios-GIM/foruster`. The anti-forensic dataset used for recall testing is permanently archived on Zenodo [16]. All materials can be accessed without restriction for research and educational purposes.

References

[1] B. D. Carrier, "Open source digital forensics tools: The legal argument," in *Proceedings of the 2003 Digital Forensics Research Workshop (DFRWS)*, 2003. [Online]. Available: https://dfrws.org/presentation/open-source-digital-forensics-tools-the-legal-argument/

[2] B. Adedayo and Z. Dlamini, "A review of volatile memory forensics for incident response," *IEEE Access*, vol. 10, pp. 55 462–55 483, 2022.

[3] E. Casey and B. Schatz, "The impact of full disk encryption on digital forensics," *IT Professional*, vol. 13, no. 3, pp. 13–20, 2011.

[4] F. Roussel, S. O'Shaughnessy, K. Djemame, and G.-G. Richardson, "Digital forensic investigation and the digital evidence life cycle," *Journal of Forensic Sciences*, vol. 61, no. 5, pp. 1173–1182, 2016.

[5] M. K. Rogers, J. Goldman, R. Mislan, T. Wedge, and S. Debrota, "Computer forensics field triage process model," *Journal of Digital Forensics, Security and Law*, vol. 1, no. 2, pp. 27–40, 2006.

[6] C. Altheide and H. Carvey, "Live forensics: A proactive approach to remote evidence collection," in *2007 Information Assurance Workshop*. IEEE, 2007, pp. 225–231.

[7] D. Brezinski and T. Killalea, "On the technical, legal, and ethical challenges of live response," Lindenhurst, IL, Tech. Rep., 2005. [Online]. Available: https://www.semanticscholar.org/paper/On-the-Technical%2C-Legal%2C-and-Ethical-Challenges-Brezinski-Killalea/5f583597d3122c07769ef797585f36e4dbb38031

[8] M. H. Ligh, A. Case, J. Levy, and A. Walters, *The art of memory forensics: detecting malware and threats in Windows, Linux, and Mac memory*. John Wiley & Sons, 2014.

[9] R. Jung, R. Krebbers, P. Jouvelot, A. Bizjak, A. Charguéraud, V.-A. Dang, J.-O. Gardner, A. K. Karkare, F. Kici, O. Marso *et al.*, "Rustbelt: Securing the foundations of the rust programming language," *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, pp. 1–32, 2021.

[10] Slint Developers, "Slint documentation," 2024, project Documentation. [Online]. Available: https://slint.dev/docs

[11] J. Blandy, J. Orendorff, and L. Tindall, *Programming Rust: Fast, Safe Systems Development*, 2nd ed. O'Reilly Media, 2021.

[12] B. Carrier, *File system forensic analysis.* Addison-Wesley Professional, 2005.

[13] *ISO/IEC 27037:2012 Information technology – Security techniques – Guidelines for identification, collection, acquisition, and preservation of digital evidence*, International Organization for Standardization Standard, 2012.

[14] J.-P. Aumasson, J. O'Connor, S. Neves, and Z. Wilcox-O'Hearn, "Blake3: one function, fast everywhere," *Cryptology ePrint Archive*, 2020, report 2019/1393. [Online]. Available: https://eprint.iacr.org/2019/1393

[15] S. L. Garfinkel, P. Farrell, V. Roussev, and G. Dinolt, "Bringing science to digital forensics with standardized forensic corpora," NIST, Tech. Rep. IR-7622, 2009.

[16] M. J. S. Fernández, M. Homaei, J. C. S. Núñez, and M. Gutierrez, "Foruster anti-forensic dataset v1.0," Nov. 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17611559

[17] E. Casey, M. Agar, G.-P. Brignardello, E. Connolly, G.-D. F-W, R. G-G, and V. Roussev, "A standard for digital forensic and incident response," in *Proceedings of the DFRWS EU 2018.* Elsevier, 2018.

Algorithm 1 Live File System Triage and Anomaly Detection

```
Require: PathsToScan: A set of initial directory paths for analysis.
Require: Profiles: A set of search profiles (defined by MIME types, extensions, or file categories).
Ensure: MatchedFiles: A map where keys are profile names and values are sets of matching file paths.
Ensure: SuspiciousFiles: A set of files that matched a profile and were also flagged as anomalous.
MatchedFiles ← NewConcurrentMap()
SuspiciousFiles ← NewConcurrentList()
for all path ∈ PathsToScan in parallel do        ▷ Process each starting path in a concurrent task
    batch ← NewList()
    for all entry ∈ WalkDirectoryAsynchronously(path) do
        if entry is a file then
            actualFormat ← IdentifyFromContent(entry.path)        ▷ Reads the file header once
            suspicionReason ← Null
            matchedProfilesForFile ← NewList()
                                                                  ▷ Anomaly Detection Step
            hiddenExt ← CheckDeceptivePattern(entry.path)         ▷ e.g., "archive.jpg.exe"
            if hiddenExt is not Null then
                suspicionReason ← DeceptiveExtension(hiddenExt)
            else
                expectedKind ← KindFromExtension(entry.extension)
                actualKind ← actualFormat.kind()
                if IsMismatch(expectedKind, actualKind) then
                    suspicionReason ← ContentMismatch(expectedKind, actualKind)
                end if
            end if
                                                                  ▷ Profile Matching Step
            for all profile ∈ Profiles do
                if profile.Matches(entry.path, actualFormat) then   ▷ Uses the already-read format
                    matchedProfilesForFile.Add(profile.name)
                end if
            end for
            fileResult ← (entry.path, matchedProfilesForFile, suspicionReason)
            batch.Add(fileResult)
            if batch is full then
                ProcessBatch(batch, MatchedFiles, SuspiciousFiles)
                batch.Clear()
            end if
        end if
    end for
    if batch is not empty then                                    ▷ Process the final batch
        ProcessBatch(batch, MatchedFiles, SuspiciousFiles)
    end if
end for
return (MatchedFiles, SuspiciousFiles)

procedure ProcessBatch(batch, MatchedFiles, SuspiciousFiles)
    for all (path, profileNames, reason) ∈ batch do
        if profileNames is not empty then
            for all name ∈ profileNames do
                MatchedFiles[name].Add(path)
            end for
            if reason is not Null then                    ▷ Correlation: only add if it's also a match
                SuspiciousFiles.Add((path, reason))
            end if
        end if
    end for
end procedure
```